MULTIPLAYER GAME DEVELOPMENT

Integrating Unity Relay with Lobbies

Practical Workshop: Connecting Players Through Relay Servers

Unity 6 • Multiplayer Services SDK 1.1.8 • Netcode for GameObjects

Today's Learning Objectives

Understand the Relay Service Architecture

Learn why Relay solves NAT traversal problems and how it works with Lobby

Create Relay Allocations for Hosts

Implement host-side code to allocate a Relay server and retrieve join codes

Store Relay Join Codes in Lobby Data

Share the Relay join code via Lobby's data dictionary for joining players

Connect Clients via Relay

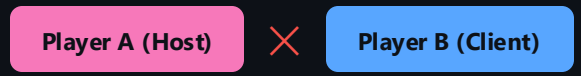
Extract the join code from Lobby data and connect to the Relay session

Why Do We Need Relay?

THE PROBLEM

Direct IP connections often fail due to NAT (Network Address Translation) and firewalls. Most home networks block incoming connections.

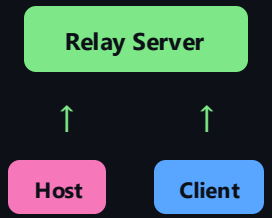
Player A wants to host a game but their router blocks incoming traffic. Player B cannot connect directly.



THE SOLUTION

Unity Relay provides a public server that both players can reach. All traffic routes through the Relay server.

Players connect outbound to the Relay. No port forwarding needed. Works through any firewall.



Lobby + Relay: The Flow

Lobby handles player discovery and matchmaking. **Relay** handles the actual network connection. The Relay join code is shared via Lobby data.

KEY INSIGHT

The Lobby is just a "meeting point" — it stores and shares data (like the Relay join code).
The actual game traffic flows through the Relay server, not through the Lobby service.

Step 1: Add Required Namespaces

Open NetworkLobbyManager.cs and add the necessary using directives at the top of the file.

You'll need these namespaces:

For Relay allocations:

```
Unity.Services.Relay
```

For Relay data models:

```
Unity.Services.Relay.Models
```

For NetworkManager and Transport:

```
Unity.Netcode
Unity.Netcode.Transports.UTP
```

YOUR TASK

Add the three using statements to your NetworkLobbyManager.cs file, alongside the existing namespaces.

💡 **HINT**

The Multiplayer Services SDK 1.1.8 includes both Lobby and Relay in one unified package. You access Relay via RelayService.Instance

📄 docs.unity.com/ugs/manual/relay/

Step 2: Create a Relay Allocation

When a player creates a lobby, they become the host. The host must request a Relay allocation — this reserves a slot on Unity's Relay server.

The Key Method:

```
RelayService.Instance
    .CreateAllocationAsync(maxConnections)
```

maxConnections = number of clients (excluding the host)

Returns an Allocation object with server connection details

For 4-player game: host + 3 clients → use maxConnections = 3

YOUR TASK

In your CreateLobby method, before calling CreateLobbyAsync, add code to create a Relay allocation.

💡 **CODE STRUCTURE**

```
Allocation allocation = await
 RelayService.Instance
 .CreateAllocationAsync(...);
```

THINK ABOUT:

Your CreateLobby method takes maxPlayers as a parameter. How do you convert this to maxConnections for Relay?

Step 3: Get the Relay Join Code

After creating an allocation, you need a join code — a unique string that clients use to connect to your Relay session.

The Key Method:

```
RelayService.Instance
    .GetJoinCodeAsync(allocation.AllocationId)
```

Uses the AllocationId from your allocation object

Returns a string — the join code for clients

This code will be stored in Lobby data for clients to retrieve

⚠ Join codes are temporary — they expire if the Relay allocation is released

YOUR TASK

After creating the allocation, call GetJoinCodeAsync to retrieve the join code string.

💡 **CODE STRUCTURE**

```
string relayJoinCode = await
 RelayService.Instance
 .GetJoinCodeAsync(...);
```

DEBUG TIP:

Add Debug.Log($"Relay Join Code: {relayJoinCode}"); to verify your code works!

Step 4: Store Join Code in Lobby

The Relay join code must be accessible to players who join the lobby. Store it in the Lobby Data dictionary.

Modify CreateLobbyOptions:

```
var
options = new CreateLobbyOptions();
options.Data = new Dictionary<...>
{
  { "RelayJoinCode",
    new DataObject(...) }
};
```

visibility: DataObject.VisibilityOptions.Member | value: join code string

YOUR TASK

Update your CreateLobbyOptions to include a Data dictionary with the "RelayJoinCode" key.

VISIBILITY OPTIONS:

Public — visible to anyone | Member — lobby members ✓ | Private — host only

💡 **HINT**

Use Member visibility — clients need to see it, but public queries shouldn't expose your join code.

Step 5: Configure Transport (Host)

The UnityTransport component needs the Relay allocation data to route traffic through the Relay server.

The Key Method:

```
NetworkManager.Singleton
  .GetComponent<UnityTransport>()
  .SetRelayServerData(
    new RelayServerData(
      allocation, "dtls")
  );
```

Connection Types:

"dtls" — Encrypted ✓ | "udp" — Unencrypted | "wss" — WebSocket

YOUR TASK

After creating the Relay allocation, configure the NetworkManager's transport with the allocation data.

PREREQUISITES:

Ensure your scene has a NetworkManager GameObject with the UnityTransport component attached.

💡 **USE "dtls"**

DTLS (Datagram Transport Layer Security) provides encryption. Always use "dtls" for production games!

📖 docs.unity.com/ugs/manual/relay/manual/relay-and-ngo

Step 6: Start the Host

After configuring the transport, call StartHost() to begin listening for client connections through the Relay.

The Complete Host Flow:

```
CreateAllocationAsync(maxConnections)
GetJoinCodeAsync(allocation.AllocationId)
SetRelayServerData(allocation, "dtls")
CreateLobbyAsync(...) // with joinCode in Data
NetworkManager.Singleton.StartHost()
```

YOUR TASK

Add NetworkManager.Singleton.StartHost() at the end of your CreateLobby method.

ORDER MATTERS!

SetRelayServerData must be called BEFORE StartHost(). The transport needs the Relay data first.

💡 TEST WITH QUANTUM CONSOLE

Use lobby.create command to test. Check Unity console for the join code and any errors.

CHECKPOINT

Host Implementation Complete

YOUR CreateLobby METHOD SHOULD NOW:

✓ Create Relay allocation ✓ Get join code ✓ Configure transport ✓ Store in Lobby Data ✓ StartHost()

TEST YOUR CODE

1. Open Quantum Console
2. Run: auth.signin YourName
3. Run: lobby.create TestLobby 4
4. Check console for Relay join code

EXPECTED OUTPUT

```
Relay Join Code: ABCD1234
 Lobby Named: TestLobby
 Players: 1/4
```

Take 15 minutes to implement and test the host side before moving on.

Step 7: Client Retrieves Join Code

When a client joins a lobby, they receive the Lobby object which contains the Data dictionary. Extract the Relay join code from there.

Look at your JoinLobby method:

```
// You already have this:
ActiveLobby = await
  LobbyService.Instance
  .JoinLobbyByIdAsync(lobbyId, options);

// Now extract the join code:
string
relayJoinCode = ActiveLobby
  .Data["RelayJoinCode"].Value;
```

⚠ Add null checking — the key might not exist if host didn't set it correctly

YOUR TASK

In your JoinLobby method (NetworkLobbyManager.cs), after joining the lobby, extract the RelayJoinCode from the Lobby Data.

SAFE ACCESS PATTERN:

```
if (lobby.Data.ContainsKey("RelayJoinCode"))
 {
 var code = lobby.Data["RelayJoinCode"].Value;
 }
```

NOTE:

See the comment in your JoinLobbyCommand:
"Step 2 Retrieve the Relay Join Code from Lobby Data"

Step 8: Join the Relay Allocation

With the Relay join code, the client can request to join the host's Relay allocation. This returns a JoinAllocation object.

The Key Method:

```
JoinAllocation
joinAllocation = await
  RelayService.Instance
  .JoinAllocationAsync(relayJoinCode);
```

JoinAllocation is similar to Allocation but for clients

Contains connection data needed by UnityTransport

Works with the same SetRelayServerData method

YOUR TASK

After extracting the relayJoinCode from Lobby data, call JoinAllocationAsync to join the host's Relay session.

💡 **CODE STRUCTURE**

```
JoinAllocation allocation = await
 RelayService.Instance
 .JoinAllocationAsync(joinCode);
```

IMPORTANT:

Both Allocation (host) and JoinAllocation (client) work with RelayServerData constructor!

Step 9: Configure Transport & Start Client

Configure the transport just like the host, then call StartClient() instead of StartHost().

The Complete Client Flow:

1. `JoinLobbyByIdAsync(lobbyId)`
2. `lobby.Data["RelayJoinCode"].Value`
3. `JoinAllocationAsync(relayJoinCode)`
4. `SetRelayServerData(joinAllocation, "dtls")`
5. `NetworkManager.Singleton.StartClient()`

✓ Same RelayServerData constructor works for both Allocation and JoinAllocation types!

YOUR TASK

Complete the JoinLobby method by adding SetRelayServerData and StartClient().

💡 CODE STRUCTURE

```
NetworkManager.Singleton
 .GetComponent<UnityTransport>()
 .SetRelayServerData(

new RelayServerData(
 joinAllocation,
"dtls"));

 NetworkManager.Singleton.StartClient();
```

HOST VS CLIENT:

Host: CreateAllocationAsync → StartHost()
Client: JoinAllocationAsync → StartClient()

TESTING

Testing with Two Unity Instances

To test multiplayer, you need two separate game instances. Use ParrelSync or build a standalone player to test alongside the Editor.

HOST INSTANCE (Editor)

- Open Quantum Console
- `auth.signin HostPlayer`
- `lobby.create MyLobby 4`
- Copy the Lobby ID from console

CLIENT INSTANCE (Build/Clone)

- Open Quantum Console
- `auth.signin ClientPlayer`
- `lobby.join [paste-lobby-id]`
- Check for "connected" message

EXPECTED CONSOLE OUTPUT (Client)

```
Joined Lobby: MyLobby (abc123...) as ClientPlayer
[Netcode] Client connected
```

Troubleshooting Common Errors

"Join code not found"
The Relay allocation expired or was never created
✓ Ensure host calls CreateAllocationAsync first

"Not authenticated" / Authentication error
Player must be signed in before using Relay
✓ Call auth.signin before lobby commands

"Failed to connect to server"
Transport not configured or mismatch in connection type
✓ Both host & client must use same "dtls"

KeyNotFoundException for "RelayJoinCode"
Host didn't store the join code in Lobby Data
✓ Check CreateLobbyOptions.Data setup

NetworkManager is null
NetworkManager not in scene or not set up
✓ Add NetworkManager + UnityTransport to scene

EXTENSION TASK

Refactoring: Create Helper Methods

Clean code is maintainable code. Create a NetworkRelayManager class to encapsulate Relay logic.

Suggested Methods:

```
// Host-side helper
async Task<string> SetupHostRelay(int maxConnections)
// Returns the join code

// Client-side helper
async Task JoinHostRelay(string relayJoinCode)
// Joins and starts client
```

BENEFITS

- ✓ Single Responsibility Principle
- ✓ Easier to test and debug
- ✓ Reusable across projects

CHALLENGE:

If you finish early, try extracting your Relay code into a separate NetworkRelayManager class.

Documentation & Resources

RELAY DOCUMENTATION

Get Started with Relay
docs.unity.com/ugs/manual/relay/manual/get-started

Relay with NGO
docs.unity.com/ugs/manual/relay/manual/relay-and-ngo

LOBBY DOCUMENTATION

Update Lobby Data
docs.unity.com/ugs/manual/lobby/manual/update-lobby-data

Game Lobby Sample
docs.unity.com/ugs/manual/lobby/manual/game-lobby-sample

NETCODE FOR GAMEOBJECTS

Using Unity Relay
docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.4/manual/relay/

SAMPLE PROJECTS

Game Lobby Sample (GitHub)
github.com/Unity-Technologies/com.unity.services.samples.game-lobby

KEY API CLASSES

`RelayService.Instance`
`Allocation` / `JoinAllocation`
`RelayServerData`

`LobbyService.Instance`
`CreateLobbyOptions` / `DataObject`
`NetworkManager.Singleton`

What You've Learned

HOST WORKFLOW

- Create Relay allocation
- Get Relay join code
- Configure UnityTransport
- Store join code in Lobby Data
- StartHost()

CLIENT WORKFLOW

- Join Lobby
- Extract join code from Lobby Data
- Join Relay allocation
- Configure UnityTransport
- StartClient()

KEY TAKEAWAY

Lobby is for discovery and data sharing. Relay is for actual network traffic. They work together — Lobby stores the Relay join code so clients know how to connect.

Questions?

Take time to complete your implementation and test with a classmate.
Don't hesitate to ask for help!

RelayService.Instance

LobbyService.Instance

NetworkManager.Singleton